Note 18: Stacks & Queues

Instructions

Our objective is to study and define the elementary data structures *stack* and *queue* using array implementations.

Background

Stacks and queues are among the most widely used linear data structures. Both can be implemented so that every fundamental method runs in $O(1)$ time.

A *stack* follows the **Last-In, First-Out (LIFO)** principle: the most recently inserted item is the first to be removed. Insertion and removal occur at the same end of the collection, called the *top*. Common applications include expression parsing, recursion management, and pattern matching.

A *queue* follows the **First-In, First-Out (FIFO)** principle: the oldest item is the first to be removed. Insertion occurs at the *back* and removal occurs at the *front*, so the two operations act on opposite ends. Common applications include task scheduling, multithreading, and print-job management.

Stack Fundamental Methods

- `push()` – Adds an item to the top of the stack.
- `pop()` – Removes the top item from the stack.
- `top()` – Returns the top item of the stack.
- `empty()` – Checks whether the stack is empty.

Queue Fundamental Methods

- `enqueue()` – Adds an item to the back of the queue.
- `dequeue()` – Removes the front item from the queue.
- `peek()` – Returns the front item of the queue.
- `empty()` – Checks whether the queue is empty.

Array Implementation of a Stack The stack uses an integer *top* that stores the *number of items currently in the stack*, together with a storage array *data*. The stack is empty when $top = 0$, and the top item is stored at index *top* - 1.

```
Push(obj)
01. if top < data.length, then
  01. data[top] ← obj
  02. top ← top + 1
02. otherwise, throw overflow error
```

```
Top()
01. if top ≠ 0, then
  01. return data[top - 1]
02. otherwise, throw out-of-range error
```

```
Pop()
01. if top ≠ 0, then
  01. top ← top - 1
```

```
Empty()
01. return top = 0
```

Array Implementation of a Queue The queue uses integer indices *front* and *back* that advance with modular arithmetic, so they wrap around the fixed-size array *data*. The queue is empty when *front* = *back*, and full when (*back* + 1) mod *data*.`length` = *front*. One slot is always left unused, so an array of size n holds at most $n - 1$ items.

```
Enqueue(obj)
01. if (back + 1) mod data.length ≠ front, then
  01. data[back] ← obj
  02. back ← (back + 1) mod data.length
02. otherwise, throw overflow error
```

```
Peek()
01. if back ≠ front, then
  01. return data[front]
02. otherwise, throw out-of-range error
```

```
Dequeue()
01. if front ≠ back, then
  01. front ← (front + 1) mod data.length
```

```
Empty()
01. return front = back
```

Example

Starting from an empty stack and an empty queue, the following sequence of operations produces the contents, *top* element, and *front* element shown below. Stacks are written with the top on the left.

Operation	Stack	Top	Queue	Front
add 1	[1]	1	[1]	1
add 2	[2, 1]	2	[1, 2]	1
add 3	[3, 2, 1]	3	[1, 2, 3]	1
remove	[2, 1]	2	[2, 3]	2
add 4	[4, 2, 1]	4	[2, 3, 4]	2
remove	[2, 1]	2	[3, 4]	3
remove	[1]	1	[4]	4
add 5	[5, 1]	5	[4, 5]	4

Tracing

For each sequence of operations below, give the contents of the stack array and its *top* value, and the contents of the queue array and its *front* value. Assume both collections start empty, and that a `remove` on an empty collection has no effect.

1. `(add 7, remove, add 3, remove, add 12, add 5)`
2. `(add 8, remove, add 17, add 8, remove, add 7, add 11, remove)`
3. `(remove, add 10, add 1, remove, add 7, remove, add 8)`
4. `(add 1, add 18, remove, add 8, remove, add 4, add 18, remove, add 12)`

Coding

Create a header file that includes only the libraries `iostream`, `string`, `sstream`, `stdexcept`, and `vector`, and defines the following classes:

- A generic *Stack* class with a private `vector` field named *data*, a private integer field named *top*, and public special member functions and fundamental methods.
- A generic *Queue* class with a private `vector` field named *data*, private integer fields named *front* and *back*, and public special member functions and fundamental methods.

Then create a `.cpp` file that tests the classes as follows:

1. Create a *Stack* object and a *Queue* object.
2. Verify that the top and front elements from your traces are correct.
3. Write the function:

   ```
   bool isPalindrome(const string& str)
   ```

   which uses a *Stack* object and a *Queue* object to return `true` if *str* is a palindrome and `false` otherwise.